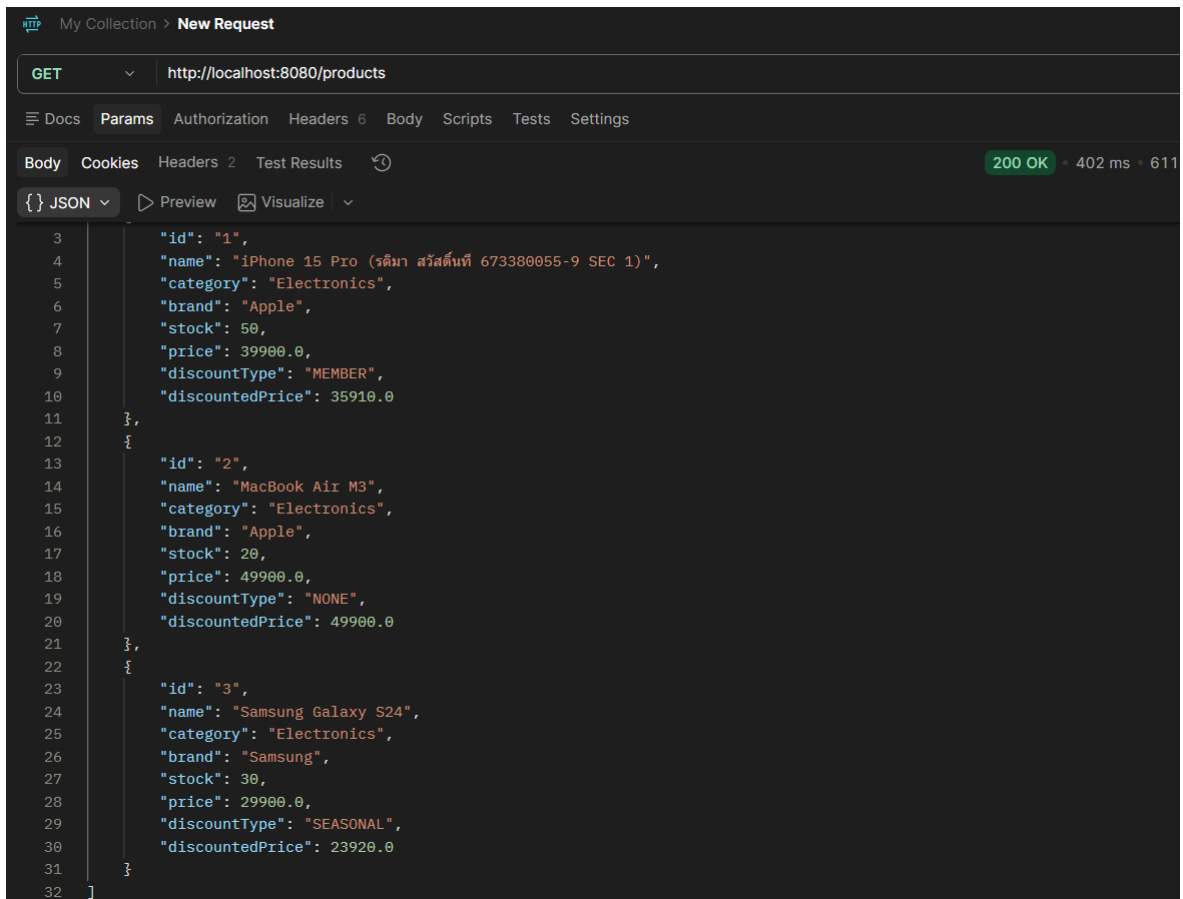


Lab 10: Spring WebFlux & WebClient

ทดสอบทุก endpoint

1.GET สินค้าทั้งหมด



```
My Collection > New Request

GET http://localhost:8080/products

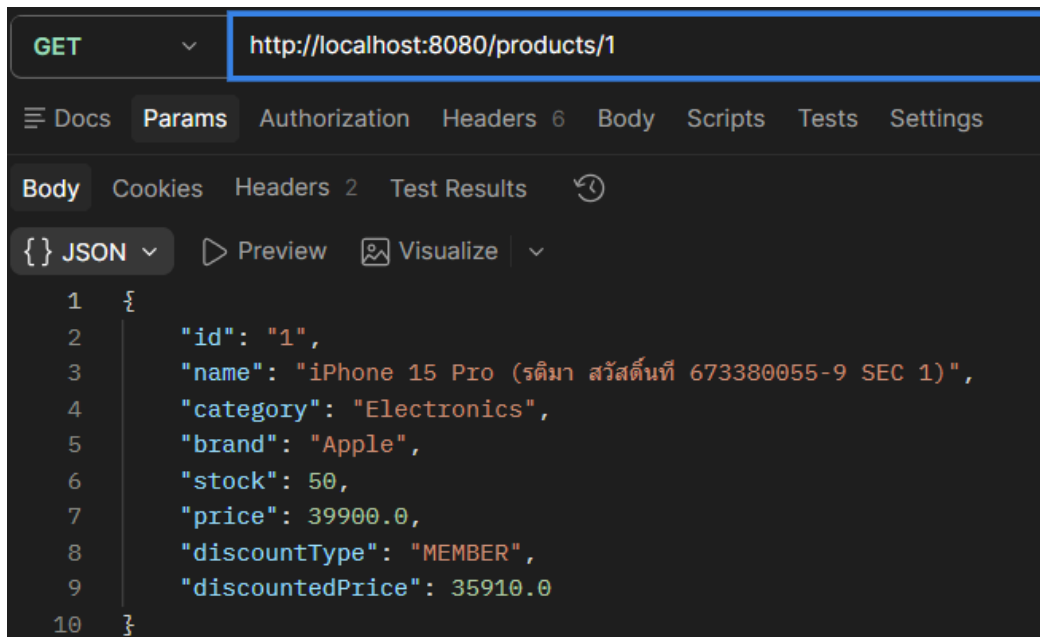
Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 2 Test Results 200 OK 402 ms 611

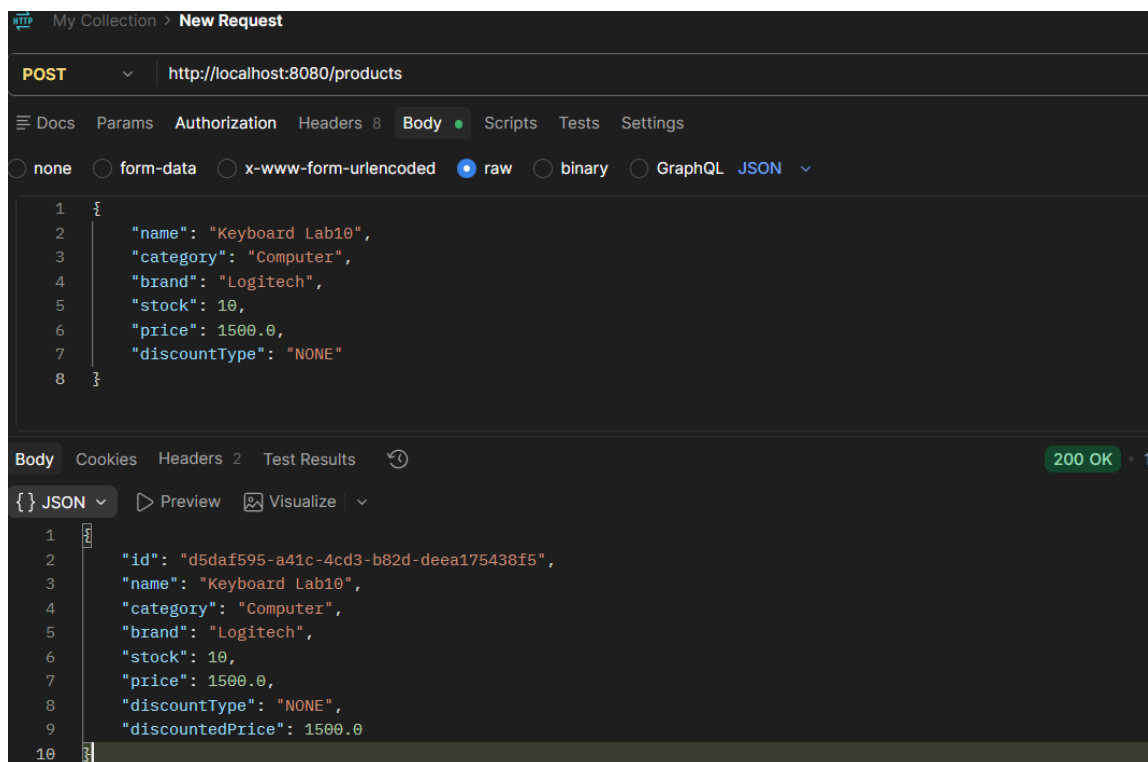
JSON Preview Visualize

3   "id": "1",
4   "name": "iPhone 15 Pro (รติมา สวัสดิ์รัตน์ 673380055-9 SEC 1)",
5   "category": "Electronics",
6   "brand": "Apple",
7   "stock": 50,
8   "price": 39900.0,
9   "discountType": "MEMBER",
10  "discountedPrice": 35910.0
11 },
12 {
13   "id": "2",
14   "name": "MacBook Air M3",
15   "category": "Electronics",
16   "brand": "Apple",
17   "stock": 20,
18   "price": 49900.0,
19   "discountType": "NONE",
20   "discountedPrice": 49900.0
21 },
22 {
23   "id": "3",
24   "name": "Samsung Galaxy S24",
25   "category": "Electronics",
26   "brand": "Samsung",
27   "stock": 30,
28   "price": 29900.0,
29   "discountType": "SEASONAL",
30   "discountedPrice": 23920.0
31 }
32 ]
```

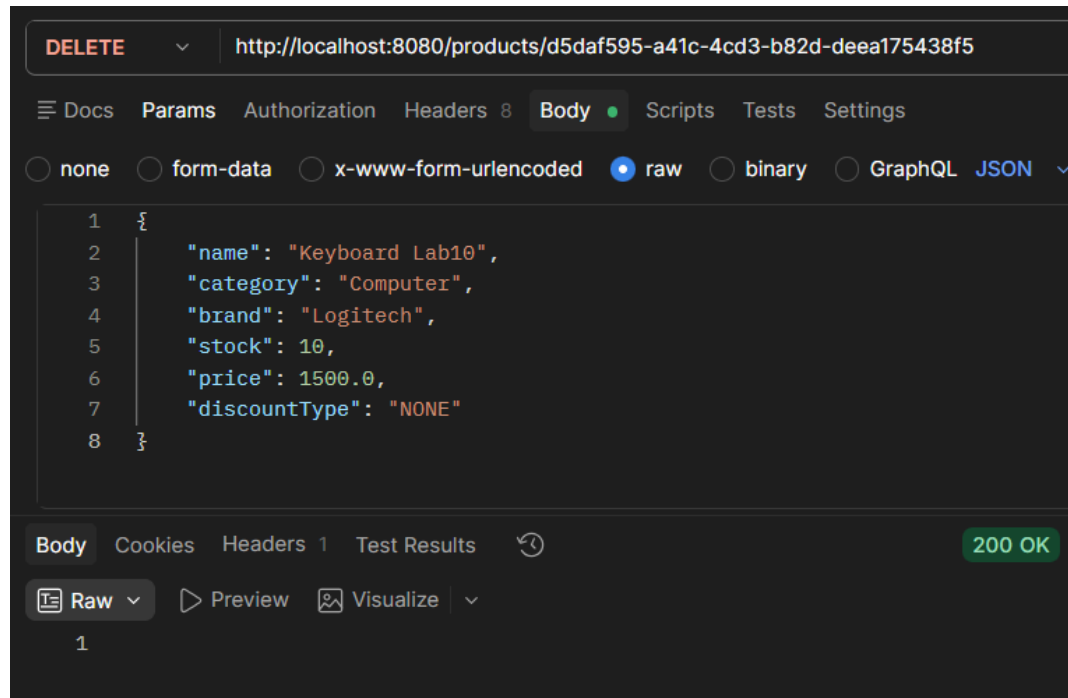
2. GET สินค้าตาม ID



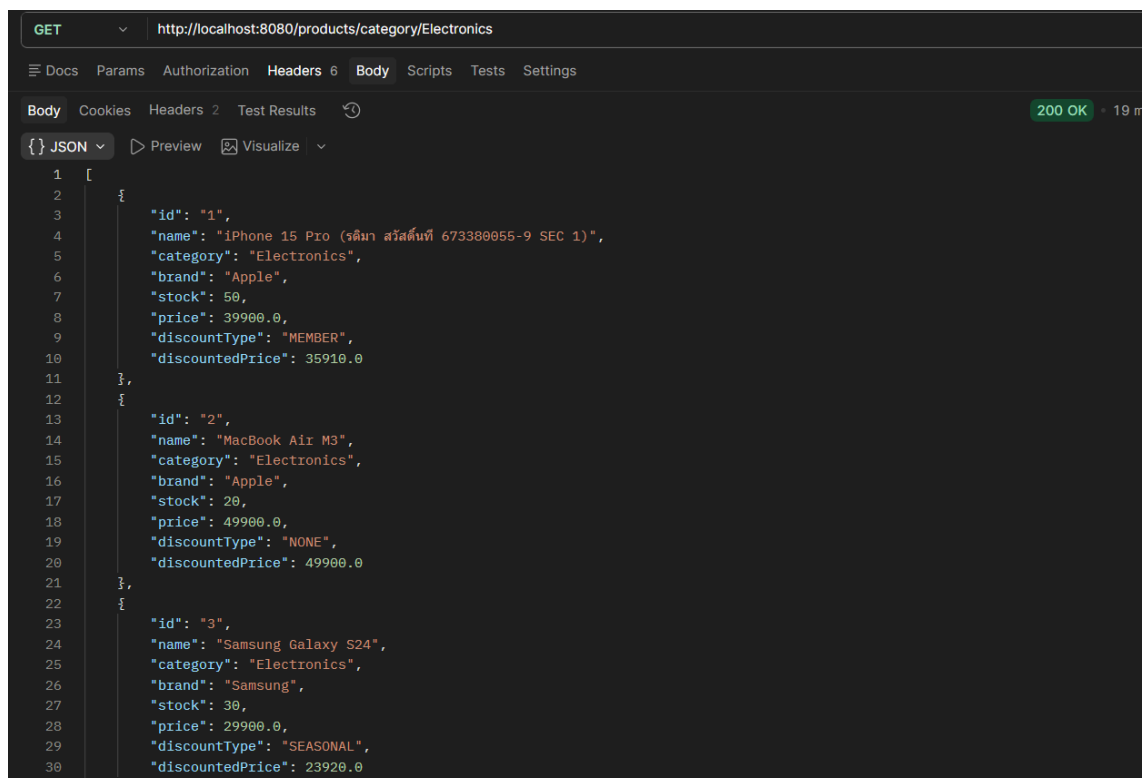
3. POST เพิ่มสินค้า



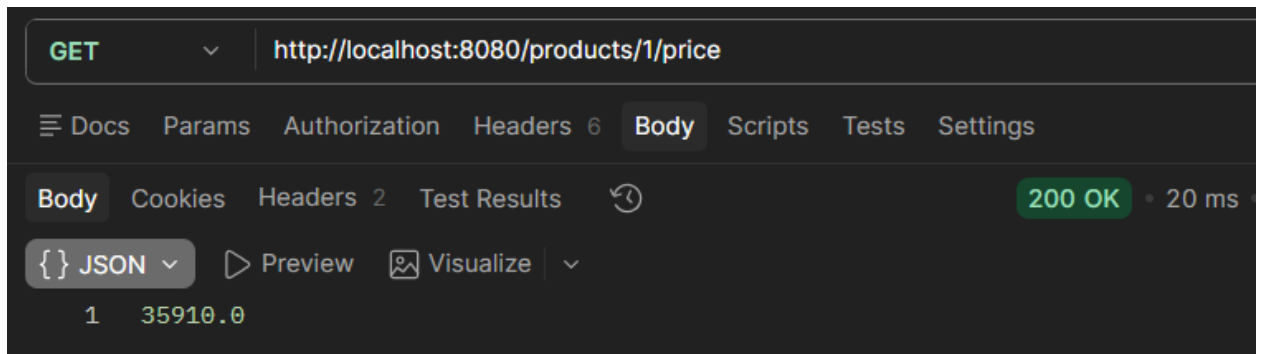
4. DELETE ลบสินค้า



5.GET ตาม Category



6.GET ราคาหลังส่วนลด



คำถามท้ายLab

1.Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ Blocking เป็นรูปแบบการทำงานที่ Thread จะต้องรอให้การดำเนินงานที่กำลังทำอยู่ เช่น การอ่านข้อมูลหรือการติดต่อกับระบบอื่นเสร็จสิ้นก่อน จึงสามารถทำงานต่อได้ ทำให้ Thread อาจถูกใช้ค้างไว้

ส่วน Reactive Programming เป็นรูปแบบการเขียนโปรแกรมที่เน้นการทำงานแบบ Asynchronous และ Non-blocking โดยข้อมูลสามารถถูกส่งผ่านเป็น Stream และระบบสามารถดำเนินงานอื่นต่อได้ระหว่างรอผลลัพธ์ ไม่ต้องรอเหมือน Blocking

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono เหมาะกับการใช้กรณี ค้นหาข้อมูล จาก ID ,บันทึกข้อมูลและลบข้อมูล, ค้นหาราคาของข้อมูล เพราะ Mono<T> ใช้แทนข้อมูลที่มีผลลัพธ์ 0 หรือ 1 ค่า

ส่วน Flux เหมาะกับการใช้กรณี ดึงข้อมูลทั้งหมด ค้นหาข้อมูลตาม Category หรือข้อมูลที่มีหลายรายการ เพราะ Flux<T> ใช้แทนข้อมูลที่มีผลลัพธ์ 0 ถึงหลายค่า

3.WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

- .get() คือ การที่เราต้องการขอข้อมูลจาก Server
- .uri() คือ การใช้กำหนด URL ที่ต้องการเรียก เช่นถ้า id = 1 จะได้/products/1
- .retrieve() คือ การใช้สำหรับรับ Response จาก Server หลังจากส่ง Request ไปแล้ว
- .bodyToMono() ใช้เมื่อ Response มีข้อมูล 0 หรือ 1 ค่า เช่น .bodyToMono(Product.class) โดยจะรับ Response แล้วแปลง JSON เช่น เป็น Product ภายใน Mono<Product>
- .bodyToFlux() ใช้เมื่อ Response มีข้อมูลหลายรายการ เช่น .bodyToFlux(Product.class) หมายถึง รับข้อมูลหลาย Product และแปลงเป็น Flux<Product>

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

- ProductRepository

Java

```
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    // ถ้าไม่พบ Product ให้คืน Mono.empty()
    if (product == null) {
        return Mono.empty();
    }

    // ถ้าพบ Product ให้ห่อข้อมูลด้วย Mono
    return Mono.just(product);
}

public Flux<Product> findAll() {
    // แปลง Collection ของ Product เป็น Flux
    return Flux.fromIterable(store.values());
}

public Mono<Product> save(Product product) {
    // บันทึก Product ลงใน Map
    store.put(product.getId(), product);

    // คืน Product ที่บันทึกในรูปแบบ Mono
    return Mono.just(product);
}

public Mono<Void> deleteById(String id) {
    // ลบ Product ตาม ID
    store.remove(id);

    // ไม่มีข้อมูลที่ต้องส่งกลับ
    return Mono.empty();
}

public Flux<Product> findByCategory(String category) {
    // ค้นหา Product ทั้งหมด แล้วกรองเฉพาะ Category ที่ตรงกัน
    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

- ProductService

Java

```
public Mono<Product> getById(String id) {
    return repository.findById(id)
        // ถ้าไม่พบ Product ให้สร้าง Error
        .switchIfEmpty(
            Mono.error(
                new RuntimeException("Product not found: " + id)
            )
        );
}

public Flux<Product> getAll() {
    // เรียก Repository เพื่อดึง Product ทั้งหมด
    return repository.findAll();
}

public Mono<Product> save(Product product) {

    // ถ้า Product ยังไม่มี ID จะทำการให้สร้าง UUID ใหม่
    if (product.getId() == null) {
        product.setId(java.util.UUID.randomUUID().toString());
    }

    return repository.save(product);
}

public Mono<Void> delete(String id) {
    // ส่งต่อการลบไปยัง Repository
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    // ค้นหา Product ตาม Category
    return repository.findByCategory(category);
}

// ใช้ Mono<Double> เพราะเราไม่ได้ต้องการรายละเอียดทั้งหมดของ Product แต่ต้องการเฉพาะราคา
public Mono<Double> getDiscountedPrice(String id) {

    return getById(id)
        // แปลง Product เป็นราคาหลังส่วนลด
        .map(p -> p.getDiscountedPrice());
}
```

- ProductController

Java

@GetMapping

```
public Flux<Product> getAll() {  
    // เรียก Service เพื่อดึง Product ทั้งหมด  
    return service.getAll();  
}
```

@PostMapping

```
public Mono<Product> save(@RequestBody Product product) {  
    // รับ Product จาก Request Body  
    // แล้วส่งให้ Service บันทึก  
    return service.save(product);  
}
```

@DeleteMapping("/{id}")

```
public Mono<Void> delete(@PathVariable String id) {  
    // รับ ID จาก URL แล้วส่งให้ Service ลบ  
    return service.delete(id);  
}
```

@GetMapping("/category/{category}")

```
public Flux<Product> getByCategory(  
    @PathVariable String category) {  
  
    // ค้นหา Product ตาม Category  
    return service.getByCategory(category);  
}
```

@GetMapping("/{id}/price")

```
public Mono<Double> getDiscountedPrice(  
    @PathVariable String id) {  
  
    // ขอราคาล้างส่วนลดของ Product  
    return service.getDiscountedPrice(id);  
}
```

@GetMapping("/{id}")

```
public Mono<Product> getById(@PathVariable String id) {  
    // ค้นหา Product ตาม ID  
    return service.getById(id);  
}
```